

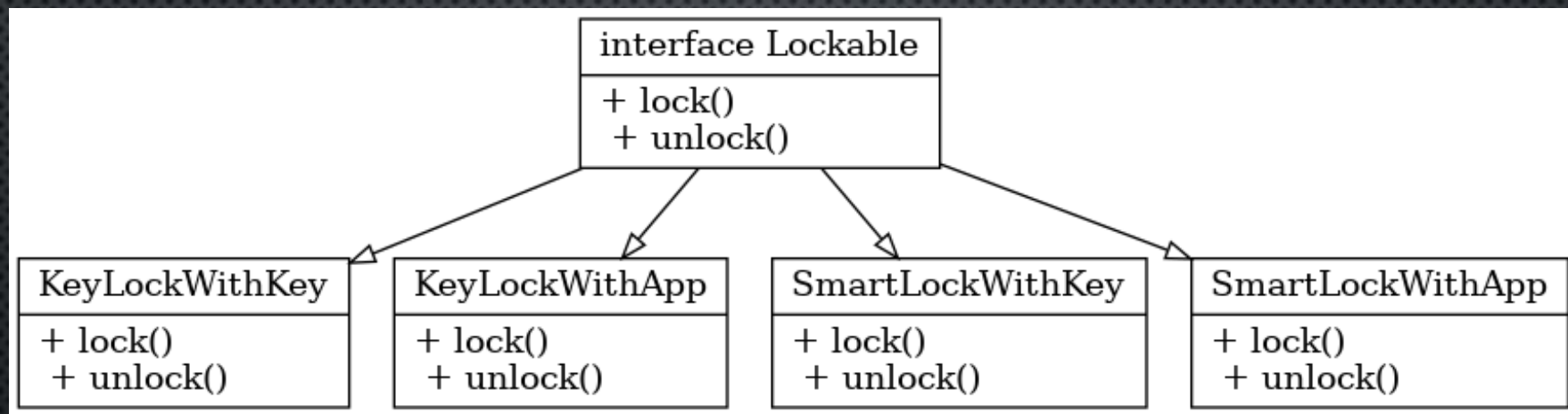
ООП 2

СТРУКТУРНИ ШАБЛОНИ ЗА ДИЗАЙН 2 ЧАСТ

ШАБЛОН МОСТ. (BRIDGE PATTERN)

- ПРОБЛЕМ
 - АВТОМОБИЛНИ ЗАКЛЮЧВАЩИ СИСТЕМИ (KEYLOCK, SMARTLOCK) И РАЗЛИЧНИ МЕХАНИЗМИ (КЛЮЧ, МОБИЛНО ПРИЛОЖЕНИЕ, ОТПЕЧАТЪК).
 - СЪЗДАВАМЕ ОТДЕЛЕН КЛАС ЗА ВСЯКА КОМБИНАЦИЯ, КОЕТО ВОДИ ДО ДУБЛИРАНЕ И ТВЪРДА ВРЪЗКА.

ШАБЛОН МОСТ. (BRIDGE PATTERN)



ШАБЛОН МОСТ. (BRIDGE PATTERN)

```
CLASS KEYLOCKWITHKEY IMPLEMENTS LOCKABLE {  
    PUBLIC VOID LOCK() {  
        SYSTEM.OUT.PRINTLN("LOCKING KEY LOCK WITH PHYSICAL KEY");  
    }  
    PUBLIC VOID UNLOCK() {  
        SYSTEM.OUT.PRINTLN("UNLOCKING KEY LOCK WITH PHYSICAL KEY");  
    }  
}
```

```
CLASS SMARTLOCKWITHAPP IMPLEMENTS LOCKABLE {  
    PUBLIC VOID LOCK() {  
        SYSTEM.OUT.PRINTLN("LOCKING SMART LOCK WITH MOBILE APP");  
    }  
    PUBLIC VOID UNLOCK() {  
        SYSTEM.OUT.PRINTLN("UNLOCKING SMART LOCK WITH MOBILE APP");  
    }  
}
```

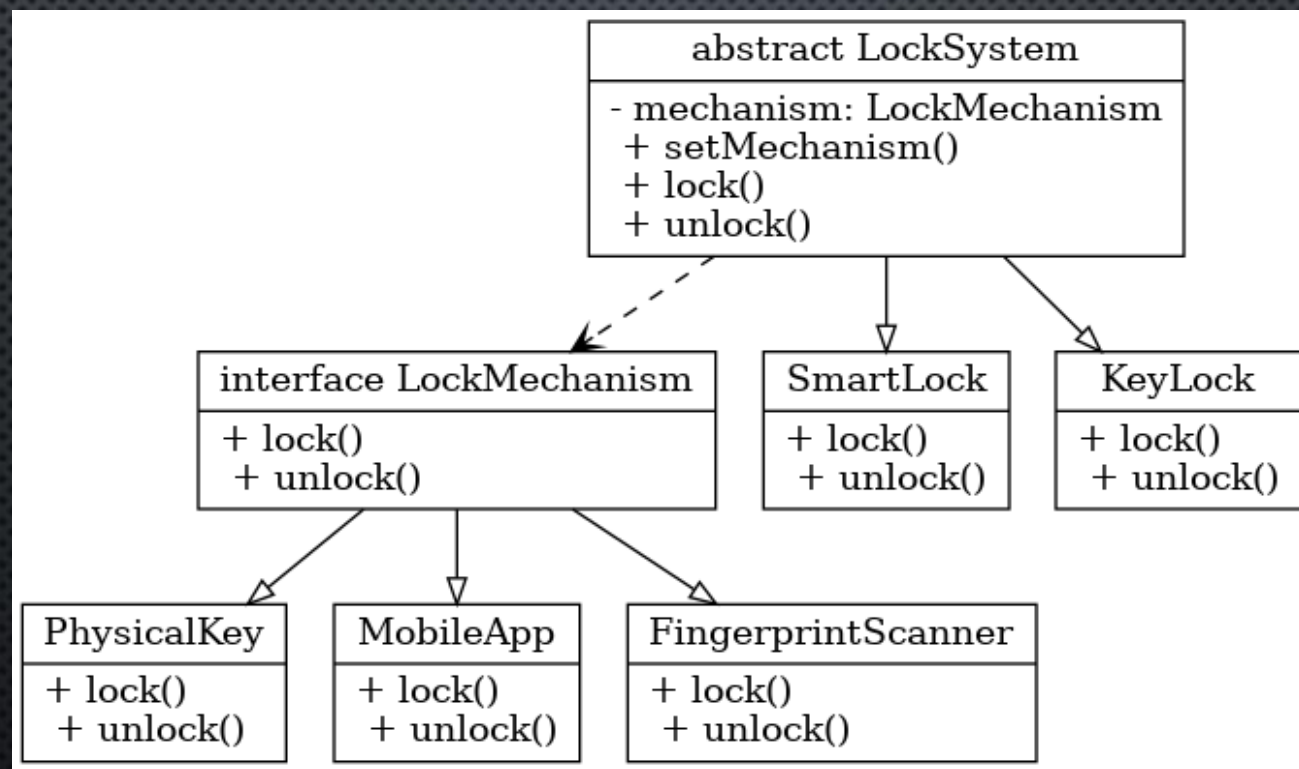
ШАБЛОН МОСТ. (BRIDGE PATTERN)

- НЕДОСТАТЪЦИ:
 - При 2 типа заключване и 3 механизма → 6 различни класа
 - При 3 типа заключване и 4 механизма → ще имаме 12 класа
 - Всеки път, когато добавим нов механизъм или тип система, броят на класовете расте експоненциално
 - Всички класове имат сходна структура и повтарящ се код
 - Механизмите не могат да се сменят по време на изпълнение (runtime)

ШАБЛОН МОСТ. (BRIDGE PATTERN)

- РЕШЕНИЕ:
 - Създаваме интерфейс `LockMechanism` с методи `lock()` и `unlock()`
 - Създаваме абстрактен клас `LockSystem`, който делегира към `LockMechanism`
 - Реализациите на `LockSystem` (напр. `SmartLock`, `KeyLock`) използват различни механизми без да ги съдържат директно

ШАБЛОН МОСТ. (BRIDGE PATTERN)



ШАБЛОН МОСТ. (BRIDGE PATTERN)

```
INTERFACE LOCKMECHANISM {  
    VOID LOCK();  
    VOID UNLOCK();  
}
```

```
ABSTRACT CLASS LOCKSYSTEM {  
    PROTECTED LOCKMECHANISM MECHANISM;  
    PUBLIC LOCKSYSTEM(LOCKMECHANISM MECHANISM) {  
        THIS.MECHANISM = MECHANISM;  
    }  
    PUBLIC VOID SETMECHANISM(LOCKMECHANISM MECHANISM) {  
        THIS.MECHANISM = MECHANISM;  
    }  
    PUBLIC ABSTRACT VOID LOCK();  
    PUBLIC ABSTRACT VOID UNLOCK();  
}
```

ШАБЛОН МОСТ. (BRIDGE PATTERN)

```
CLASS PHYSICALKEY IMPLEMENTS LOCKMECHANISM {  
    PUBLIC VOID LOCK() {  
        SYSTEM.OUT.PRINTLN("LOCKING WITH PHYSICAL KEY");  
    }  
    PUBLIC VOID UNLOCK() {  
        SYSTEM.OUT.PRINTLN("UNLOCKING WITH PHYSICAL KEY");  
    }  
}
```

```
CLASS MOBILEAPP IMPLEMENTS LOCKMECHANISM {  
    PUBLIC VOID LOCK() {  
        SYSTEM.OUT.PRINTLN("LOCKING VIA MOBILE APP");  
    }  
    PUBLIC VOID UNLOCK() {  
        SYSTEM.OUT.PRINTLN("UNLOCKING VIA MOBILE APP");  
    }  
}
```

```
}  
}  
  
CLASS SMARTLOCK EXTENDS LOCKSYSTEM {  
    PUBLIC SMARTLOCK(LOCKMECHANISM MECHANISM) {  
        SUPER(MECHANISM);  
    }  
    PUBLIC VOID LOCK() {  
        MECHANISM.LOCK();  
    }  
    PUBLIC VOID UNLOCK() {  
        MECHANISM.UNLOCK();  
    }  
}
```

ШАБЛОН МОСТ. (BRIDGE PATTERN)

```
LOCKSYSTEM LOCKSYSTEM = NEW SMARTLOCK(NEW PHYSICALKEY());
```

```
LOCKSYSTEM.UNLOCK();
```

```
LOCKSYSTEM.SETMECHANISM(NEW MOBILEAPP());
```

```
LOCKSYSTEM.UNLOCK();
```

ШАБЛОН МОСТ. (BRIDGE PATTERN)

- ИЗБЯГВА ЕКСПЛОЗИЯТА ОТ КЛАСОВЕ ПРИ ВСЯКА КОМБИНАЦИЯ
- УЛЕСНЯВА РАЗШИРЯВАНЕ ПО ДВЕТЕ ОСИ: ТИПОВЕ ЗАКЛЮЧВАНЕ И МЕХАНИЗМИ
- ПОЗВОЛЯВА ПОДМЯНА НА РЕАЛИЗАЦИЯ В RUNTIME
- ПОДДЪРЖА OPEN/CLOSED ПРИНЦИПА
- ГЪВКАВОСТ И ПОВТОРНА УПОТРЕБА НА КОД

ОЛЕКОТЯВАЩ ШАБЛОН (FLYWEIGHT PATTERN)

- ООП – РЕФЛЕКЦИЯ НА РЕАЛНИЯ СВЯТ
 - ОБЕКТИ
 - МНОГО ОБЕКТИ
- КОНТРОЛ ПО СЪЗДАВАНЕТО

ОЛЕКОТЯВАЩ ШАБЛОН

- ПРОБЛЕМ
 - ON-LINE СРЕДА ЗА ПРОГРАМИРАНЕ

ОЛЕКОТЯВАЩ ШАБЛОН

- ```
PUBLIC CLASS CODE {
 PRIVATE STRING CODE;
 PUBLIC STRING GETCODE() {
 RETURN CODE;
 }
 PUBLIC VOID SETCODE(STRING CODE) {
 THIS.CODE = CODE;
 }
}
```

# ОЛЕКОТЯВАЩ ШАБЛОН

- ```
PUBLIC INTERFACE PLATFORM {  
    PUBLIC VOID EXECUTE(CODE CODE);  
}
```

ОЛЕКОТЯВАЩ ШАБЛОН

- ```
PUBLIC CLASS JAVAPLATFORM IMPLEMENTS PLATFORM {
 PUBLIC JAVAPLATFORM(){
 SYSTEM.OUT.PRINTLN("JAVAPLATFORM OBJECT CREATED");
 }
 @OVERRIDE
 PUBLIC VOID EXECUTE(CODE CODE) {
 SYSTEM.OUT.PRINTLN("COMPILING AND EXECUTING JAVA CODE.");
 }
}
```

# ОЛЕКОТЯВАЩ ШАБЛОН

- `PLATFORM PLATFORM = NEW JAVAPLATFORM();`  
`PLATFORM.EXECUTE(CODE);`
- ПРОБЛЕМ?

# ОЛЕКОТЯВАЩ ШАБЛОН

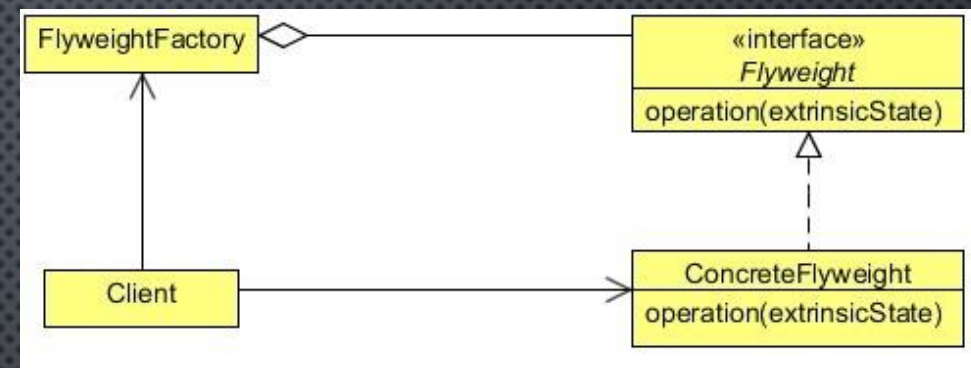
- СПОДЕЛЯНЕ НА ОБЕКТИ — ПО ВИСОКА ЕФЕКТИВНОСТ
  - ИЗПОЛЗВАНЕ В МНОЖЕСТВО КОНТЕКСТИ ЕДНОВРЕМЕННО
- КЛЮЧОВАТА КОНЦЕПЦИЯ Е РАЗГРАНИЧЕНИЕТО МЕЖДУ ВЪТРЕШНО И ВЪНШНО СЪСТОЯНИЕ
  - ВЪТРЕШНО — ОБЕКТА ЗА СПОДЕЛЯНЕ (ЧАСТ ОТ ВЪТРЕШНАТА ИМПЛЕМЕНТАЦИЯ НА ШАБЛОНА)
  - ВЪНШНО — ИДВА ОТ КЛИЕНТА (ПОЛЗВАТЕЛЯ. НЕ СЕ СПОДЕЛЯ)

# ОЛЕКОТЯВАЩ ШАБЛОН

- СЪЗДАВАНЕ НА ГОЛЯМ БРОЙ ОБЕКТИ, КОИТО СА УНИКАЛНИ САМО ПО ОТНОШЕНИЕ НА НЯКОЛКО ПАРАМЕТЪРА - СЪДЪРЖАТ НЯКОИ ВЪТРЕШНИ, ИНВАРИАНТНИ ДАННИ, КОИТО СА ОБЩИ ЗА ВСИЧКИ ОБЕКТИ.
  - СЪЗДАВАНЕ И ПОДДЪРЖАНЕ НА ГОЛЯМА ГРУПА НА ТАКИВА ОБЕКТИ - МНОГО СКЪПО ПО ОТНОШЕНИЕ НА ИЗПОЛЗВАНЕ НА ПАМЕТ И ПРОИЗВОДИТЕЛНОСТ

# ОЛЕКОТЯВАЩ ШАБЛОН

- FLYWEIGHT - ДЕКЛАРИРА ИНТЕРФЕЙС, ЧРЕЗ КОЙТО ОЛЕКОТЕНИЯ МОДЕЛ МОЖЕ ДА ПРИЕМА И ДЕЙСТВА ВЪВ ВЪНШНО СЪСТОЯНИЕ



- CONCRETEFLYWEIGHT - РЕАЛИЗИРА ИНТЕРФЕЙСА FLYWEIGHT И ДОБАВЯ СЪХРАНЕНИЕ ЗА ВЪТРЕШНО СЪСТОЯНИЕ.
  - CONCRETEFLYWEIGHT ТРЯБВА ДА МОЖЕ ДА СЕ СПОДЕЛЯ.
  - ВСЯКО СЪСТОЯНИЕ, КОЕТО СЪХРАНЯВА, ТРЯБВА ДА БЪДЕ ВЪТРЕШНО- ДА БЪДЕ НЕЗАВИСИМ ОТ КОНТЕКСТА В КОЙТО СЕ ИЗПОЛЗВА.
- FLYWEIGHTFACTORY - СЪЗДАВА И УПРАВЛЯВА ОЛЕКОТЕНИ ОБЕКТИ
  - ГАРАНТИРА, ЧЕ ОЛЕКОТЕНИЯ ОБЕКТ СЕ СПОДЕЛЯ ПРАВИЛНО.
  - ПРЕДОСТАВЯ СЪЩЕСТВУВАЩ ЕКЗЕМПЛЯР ИЛИ СЪЗДАВА ТАКЪВ, АКО НЕ СЪЩЕСТВУВА
- CLIENT
  - ПОДДЪРЖА РЕФЕРЕНЦИЯ КЪМ ОЛЕКОТЕНИЯ МОДЕЛ
  - ИЗЧИСЛЯВА ИЛИ СЪХРАНЯВА ВЪНШНОТО СЪСТОЯНИЕ НА ОЛЕКОТЕНИЯ МОДЕЛ

# ОЛЕКОТЯВАЩ ШАБЛОН

```
import java.util.HashMap;

import java.util.Map;

public final class PlatformFactory {

 private static Map<String, Platform> map = new
 HashMap<>();

 private PlatformFactory(){
 throw new AssertionError("Cannot instantiate the class");
 }

 public static synchronized Platform
getPlatformInstance(String platformType){

 Platform platform = map.get(platformType);

 if(platform==null){

 switch(platformType){

 case "C" : platform = new CPlatform();
```

```
 break;

 case "CPP" : platform = new CPPPlatform();

 break;

 case "Java" : platform = new JavaPlatform();

 break;

 case "Ruby" : platform = new RubyPlatform();

 break;

 }

 map.put(platformType, platform);

 }

 return platform;

 }

}
```

# ОЛЕКОТЯВАЩ ШАБЛОН

```
PUBLIC CLASS TESTFLYWEIGHT {
 PUBLIC STATIC VOID MAIN(String[] ARGS) {
 CODE CODE = NEW CODE();
 CODE.SETCODE("C CODE...");

 PLATFORM PLATFORM =
 PLATFORMFACTORY.GETPLATFORMINSTANCE("C");
 PLATFORM.EXECUTE(CODE);
 SYSTEM.OUT.PRINTLN("*****");
 CODE = NEW CODE();
```

```
 CODE.SETCODE("C CODE2...");
 PLATFORM = PLATFORMFACTORY.GETPLATFORMINSTANCE("C");
 PLATFORM.EXECUTE(CODE);
 SYSTEM.OUT.PRINTLN("*****");

 CODE = NEW CODE();
 CODE.SETCODE("JAVA CODE...");
 PLATFORM = PLATFORMFACTORY.GETPLATFORMINSTANCE("JAVA");
 PLATFORM.EXECUTE(CODE);
 SYSTEM.OUT.PRINTLN("*****");
```

# ОЛЕКОТЯВАЩ ШАБЛОН

```
CODE = NEW CODE();
```

```
CODE.SETCODE("JAVA CODE2...");
```

```
PLATFORM = PLATFORMFACTORY.GETPLATFORMINSTANCE("JAVA"); CODE.SETCODE("RUBY CODE2...");
```

```
PLATFORM.EXECUTE(CODE);
```

```
SYSTEM.OUT.PRINTLN("*****");
```

```
CODE = NEW CODE();
```

```
CODE.SETCODE("RUBY CODE...");
```

```
PLATFORM = PLATFORMFACTORY.GETPLATFORMINSTANCE("RUBY");
```

```
PLATFORM.EXECUTE(CODE);
```

```
SYSTEM.OUT.PRINTLN("*****");
```

```
CODE = NEW CODE();
```

```
PLATFORM = PLATFORMFACTORY.GETPLATFORMINSTANCE("RUBY");
```

```
PLATFORM.EXECUTE(CODE);
```

```
}
```

```
}
```

# ОЛЕКОТЯВАЩ ШАБЛОН

CPLATFORM OBJECT CREATED

COMPILING AND EXECUTING C CODE.

\*\*\*\*\*

COMPILING AND EXECUTING C CODE.

\*\*\*\*\*

JAVAPLATFORM OBJECT CREATED

COMPILING AND EXECUTING JAVA CODE.

\*\*\*\*\*

COMPILING AND EXECUTING JAVA CODE.

\*\*\*\*\*

RUBYPLATFORM OBJECT CREATED

COMPILING AND EXECUTING RUBY CODE.

\*\*\*\*\*

COMPILING AND EXECUTING RUBY CODE.

# ОЛЕКОТЯВАЩ ШАБЛОН

- 2К ПОТРЕБИТЕЛИ ЕДНОВРЕМЕННО ИЗПОЛЗВАТ САЙТА, ТОЧНО 2К ОЛЕКОТЕНИ ОБЕКТИ ЩЕ СЕ СЪЗДАВАТ И САМО 4 ТЕЖКИ ПЛАТФОРМЕНИ ОБЕКТИ ЩЕ СЕ ИНСТАНЦИРАТ
  - АКО СЕ ПОЛЗВАТ ВСИЧКИТЕ ПЛАТФОРМИ

# ОЛЕКОТЯВАЩ ШАБЛОН

- РАЗХОДИТЕ ЗА СЪХРАНЕНИЕ СА ВИСОКИ – МНОГО ОБЕКТИ, МНОГО ПАМЕТ
- ПОВЕЧЕТО СЪСТОЯНИЯ НА ОБЕКТА МОГАТ ДА БЪДАТ НАПРАВЕНИ ВЪНШНИ.
- МНОГО ГРУПИ ОТ ОБЕКТИ МОГАТ ДА БЪДАТ ЗАМЕНЕНИ ОТ ОТНОСИТЕЛНО МАЛКО СПОДЕЛЕНИ ОБЕКТИ – СЛЕД КАТО ВЪНШНОТО СЪСТОЯНИЕ БЪДЕ ПРЕМАХНАТО.
- ПРИЛОЖЕНИЕТО НЕ ЗАВИСИ ОТ САМОЛИЧНОСТТА НА ОБЕКТА

# ШАБЛОН ПРОКСИ (PROXY PATTERN)

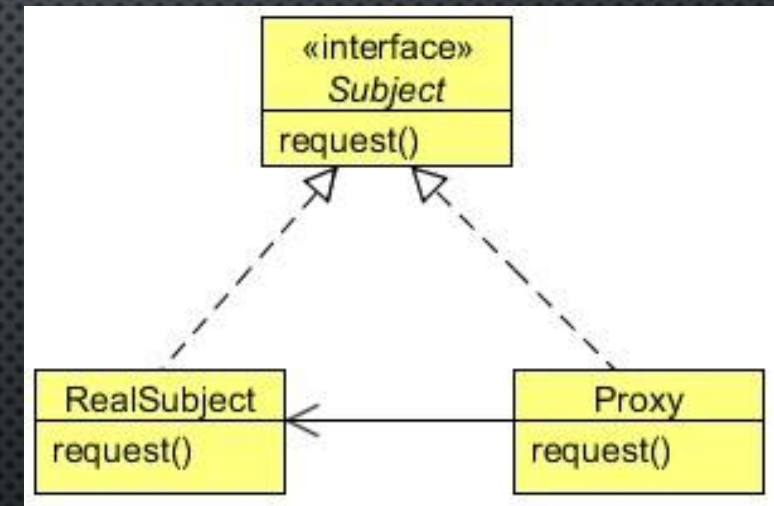
- ДОСТЪП ДО ОБЕКТ (ОТДАЛЕЧЕН)
  - СКЪП ЗА СЪЗДАВАНЕ
  - СИГУРНОСТ
- ДОСТЪП ДО РАЗЛИЧНА JVM
- РАЗЛИЧНИ ПРАВА ЗА ДОСТЪП

# ШАБЛОН ПРОКСИ (PROXY PATTERN)

- В ПРОКСИ МОДЕЛА КЛИЕНТЪТ НЕ ГОВОРИ ДИРЕКТНО С ОРИГИНАЛНИЯ ОБЕКТ - ДЕЛЕГИРА ИЗВИКВАНИЯТА НА ПРОКСИ ОБЕКТА, КОЙТО ИЗВИКВА МЕТОДИ НА ОРИГИНАЛНИЯ ОБЕКТ
  - КЛИЕНТЪТ НЕ ЗНАЕ ЗА ПРОКСИТО - ПРОКСИТО ДЕЙСТВА КАТО ОРИГИНАЛЕН ОБЕКТ ЗА КЛИЕНТА
  - ИМА МНОГО ВАРИАЦИИ НА ТОЗИ ПОДХОД

# ШАБЛОН ПРОКСИ (PROXY PATTERN)

- PROXY – ИМА РЕФЕРЕНЦИЯ КЪМ РЕАЛНИЯ ОБЕКТ REALSUBJECT (ИЛИ КЪМ ИНТЕРГФЕЙСА SUBJECT)
  - ПРЕДОСТАВЯ ИНТЕРФЕЙС, ИДЕНТИЧЕН С ТОЗИ НА SUBJECT, ТАКА ЧЕ ДА МОЖЕ ДА БЪДЕ ЗАМЕНЕН С РЕАЛНИЯ ОБЕКТ. 1в.
  - КОНТРОЛИРА ДОСТЪПА ДО РЕАЛНИЯ ОБЕКТ И МОЖЕ ДА БЪДЕ ОТГОВОРЕН ЗА СЪЗДАВАНЕТО И ИЗТРИВАНЕТО МУ
- SUBJECT - ДЕФИНИРА ОБЩИЯ ИНТЕРФЕЙС ЗА REALSUBJECT И PROXY, ТАКА ЧЕ ПРОКСИТО ДА МОЖЕ ДА СЕ ИЗПОЛЗВА НАВСЯКЪДЕ, КЪДЕТО Е ОЧАКВАН REALSUBJECT
- REALSUBJECT - ДЕФИНИРА РЕАЛНИЯ ОБЕКТ, КОЙТО ПРОКСИТО ПРЕДСТАВЛЯВА



# ШАБЛОН ПРОКСИ (PROXY PATTERN)

- ТРИ ОСНОВНИ ВАРИАНТА НА ПРОКСИ МОДЕЛА:
  - ОТДАЛЕЧЕНО ПРОКСИ – ПРЕДОСТАВЯ ЛОКАЛЕН ПРЕДСТАВИТЕЛ ЗА ОБЕКТ В РАЗЛИЧНО АДРЕСНО ПРОСТРАНСТВО.
  - ВИРТУАЛНО ПРОКСИ СЪЗДАВА “СКЪПИ” ОБЕКТИ ПРИ ПОИСКВАНЕ.
  - ПРОКСИ ЗА ЗАЩИТА – КОНТРОЛИРА ДОСТЪПА ДО ОРИГИНАЛНИЯ ОБЕКТ. ПОЛЕЗНИ, КОГАТО ОБЕКТИТЕ ТРЯБВА ДА ИМАТ РАЗЛИЧНИ ПРАВА ЗА ДОСТЪП

# ШАБЛОН ПРОКСИ (ОТДАЛЕЧЕНО(REMOTE))

- КОМПАНИЯ ЗА ПИЦИ
  - ЕЖЕДНЕВНИ ОТЧЕТИ ОТ КЛОНОВЕТЕ
  - DESKTOP — REMOTE JVM

# ШАБЛОН ПРОКСИ (ОТДАЛЕЧЕНО(REMOTE))

```
IMPORT JAVA.RMI.REMOTE;
```

```
IMPORT JAVA.RMI.REMOTEEXCEPTION;
```

```
PUBLIC INTERFACE REPORTGENERATOR EXTENDS REMOTE{
```

```
 PUBLIC STRING GENERATEDAILYREPORT() THROWS REMOTEEXCEPTION;
```

```
}
```

# ШАБЛОН ПРОКСИ (ОТДАЛЕЧЕНО(REMOTE))

```
import java.rmi.Naming;

import java.rmi.RemoteException;

import java.rmi.server.UnicastRemoteObject;

import java.util.Date;

public class ReportGeneratorImpl extends UnicastRemoteObject implements
ReportGenerator{

 private static final long serialVersionUID = 3107413009881629428L;

 protected ReportGeneratorImpl() throws RemoteException {

 }
```

```
@Override

public String generateDailyReport() throws RemoteException {

 StringBuilder sb = new StringBuilder();

 sb.append("*****LOCATION X DAILY REPORT*****");

 sb.append("\n Location ID: 012");

 sb.append("\n Today's Date: "+new Date());

 sb.append("\n Total Pizza Sell: 112");

 sb.append("\n Total Sale: $2534");

 sb.append("\n Net Profit: $1985");

 sb.append("\n *****");

 return sb.toString();

}
```

# ШАБЛОН ПРОКСИ (ОТДАЛЕЧЕНО(REMOTE))

```
PUBLIC STATIC VOID MAIN(STRING[] ARGS) {
 TRY {
 REPORTGENERATOR REPORTGENERATOR = NEW REPORTGENERATORIMPL();
 NAMING.REBIND("PIZZACOREMOTEGENERATOR", REPORTGENERATOR);
 } CATCH (EXCEPTION E) {
 E.PRINTSTACKTRACE();
 }
}
```

# ШАБЛОН ПРОКСИ (ОТДАЛЕЧЕНО(REMOTE))

```
import java.rmi.Naming;

public class ReportGeneratorClient {

 public static void main(String[] args) {

 new ReportGeneratorClient().generateReport();

 }

 public void generateReport(){

 try {

 ReportGenerator reportGenerator = (ReportGenerator)Naming.lookup(" rmi://127.0.0.1/PizzaCoRemoteGenerator");

 System.out.println(reportGenerator.generateDailyReport());

 } catch (Exception e) {

 e.printStackTrace();

 }

 }

}
```

# ШАБЛОН ПРОКСИ (ОТДАЛЕЧЕНО(REMOTE))

\*\*\*\*\*LOCATION X DAILY REPORT\*\*\*\*\*

LOCATION ID: 012

TODAY'S DATE: MON FEB 1 20:11:23 IST 2022

TOTAL PIZZA SELL: 112

TOTAL SALE: \$2534

NET PROFIT: \$1985

\*\*\*\*\*

# ШАБЛОН ПРОКСИ (ВИРТУАЛНО(VIRTUAL))

- LAZY LOADING
  - ПРЕДИМСТВОТО НА ТОЗИ ПОДХОД Е ПО-БЪРЗОТО ВРЕМЕ ЗА СТАРТИРАНЕ НА ПРИЛОЖЕНИЕТО, ТЪЙ КАТО НЕ Е НЕОБХОДИМО ДА СЕ СЪЗДАВАТ И ЗАРЕЖДАТ ВСИЧКИ ОБЕКТИ НА ПРИЛОЖЕНИЕТО.

# ШАБЛОН ПРОКСИ (ВИРТУАЛНО(VIRTUAL))

- ПРОБЛЕМ
  - КОМПАНИЯ
  - РАБОТНИЦИ
  - СПИСЪЦИ С РАБОТНИЦИ

# ШАБЛОН ПРОКСИ (ВИРТУАЛНО(VIRTUAL))

```
PUBLIC CLASS COMPANY {
 PRIVATE STRING COMPANYNAME;
 PRIVATE STRING COMPANYADDRESS;
 PRIVATE STRING COMPANYCONTACTNO;
 PRIVATE CONTACTLIST CONTACTLIST ;

 PUBLIC COMPANY(STRING COMPANYNAME,STRING COMPANYADDRESS, STRING
 COMPANYCONTACTNO, CONTACTLIST CONTACTLIST){

 THIS.COMPANYNAME = COMPANYNAME;
 THIS.COMPANYADDRESS = COMPANYADDRESS;
 THIS.COMPANYCONTACTNO = COMPANYCONTACTNO;
 THIS.CONTACTLIST = CONTACTLIST;
 SYSTEM.OUT.PRINTLN("COMPANY OBJECT CREATED...");
 }
}
```

```
PUBLIC STRING GETCOMPANYNAME() {
 RETURN COMPANYNAME;
}

PUBLIC STRING GETCOMPANYADDRESS() {
 RETURN COMPANYADDRESS;
}

PUBLIC STRING GETCOMPANYCONTACTNO() {
 RETURN COMPANYCONTACTNO;
}

PUBLIC CONTACTLIST GETCONTACTLIST(){
 RETURN CONTACTLIST;
}
}
```

# ШАБЛОН ПРОКСИ (ВИРТУАЛНО(VIRTUAL))

```
import java.util.List;

public interface ContactList {
 public List<Employee> getEmployeeList();
}
```

# ШАБЛОН ПРОКСИ (ВИРТУАЛНО(VIRTUAL))

```
import java.util.ArrayList;
import java.util.List;

public class ContactListImpl implements ContactList {
 @Override
 public List<Employee> getEmployeeList() {
 return getEmplist();
 }

 private static List<Employee> getEmplist() {
 List<Employee> emplist = new ArrayList<Employee>(5);
 emplist.add(new Employee("Employee A", 2565.55, "SE"));
```

```
 emplist.add(new Employee("Employee B", 22574, "MANAGER"));
 emplist.add(new Employee("Employee C", 3256.77, "SSE"));
 emplist.add(new Employee("Employee D", 4875.54, "SSE"));
 emplist.add(new Employee("Employee E", 2847.01, "SE"));

 return emplist;
 }
}
```

# ШАБЛОН ПРОКСИ (ВИРТУАЛНО(VIRTUAL))

```
IMPORT JAVA.UTIL.LIST;

PUBLIC CLASS CONTACTLISTPROXYIMPL IMPLEMENTS CONTACTLIST{

PRIVATE CONTACTLIST CONTACTLIST;

@OVERRIDE

PUBLIC LIST<EMPLOYEE> GETEMPLOYEELIST() {

IF(CONTACTLIST == NULL){

SYSTEM.OUT.PRINTLN("CREATING CONTACT LIST AND FETCHING LIST OF

EMPLOYEES...");

CONTACTLIST = NEW CONTACTLISTIMPL();

}

RETURN CONTACTLIST.GETEMPLOYEELIST();

}
```

# ШАБЛОН ПРОКСИ (ВИРТУАЛНО(VIRTUAL))

```
PUBLIC CLASS EMPLOYEE {
 PRIVATE STRING EMPLOYEENAME;
 PRIVATE DOUBLE EMPLOYEESALARY;
 PRIVATE STRING EMPLOYEEDESIGNATION;
 PUBLIC EMPLOYEE(STRING EMPLOYEENAME,DOUBLE EMPLOYEESALARY,STRING -
 EMPLOYEEDESIGNATION){
 THIS.EMPLOYEENAME = EMPLOYEENAME;
 THIS.EMPLOYEESALARY = EMPLOYEESALARY;
 THIS.EMPLOYEEDESIGNATION = EMPLOYEEDESIGNATION;
 }
 PUBLIC STRING GETEMPLOYEENAME() {
 RETURN EMPLOYEENAME;
 }
}
```

```
}
 PUBLIC DOUBLE GETEMPLOYEESALARY() {
 RETURN EMPLOYEESALARY;
 }
 PUBLIC STRING GETEMPLOYEEDESIGNATION() {
 RETURN EMPLOYEEDESIGNATION;
 }
 PUBLIC STRING TOSTRING(){
 RETURN "EMPLOYEE NAME: "+EMPLOYEENAME+", EMPLOYEEDESIGNATION: "+ -
 EMPLOYEEDESIGNATION+", EMPLOYEE SALARY: "+EMPLOYEESALARY;
 }
}
```

# ШАБЛОН ПРОКСИ (ВИРТУАЛНО(VIRTUAL))

```
IMPORT JAVA.UTIL.LIST;

PUBLIC CLASS TestVIRTUALPROXY {

PUBLIC STATIC VOID MAIN(STRING[] ARGS) {

CONTACTLIST CONTACTLIST = NEW CONTACTLISTPROXYIMPL();

COMPANY COMPANY = NEW COMPANY("ABC COMPANY", „BULGARIA", "+359-284589656",
CONTACTLIST);

SYSTEM.OUT.PRINTLN("COMPANY NAME: "+COMPANY.GETCOMPANYNAME());

SYSTEM.OUT.PRINTLN("COMPANY ADDRESS: "+COMPANY.GETCOMPANYADDRESS());

SYSTEM.OUT.PRINTLN("COMPANY CONTACT No.: "+COMPANY.GETCOMPANYCONTACTNO());

SYSTEM.OUT.PRINTLN("REQUESTING FOR CONTACT LIST");

CONTACTLIST = COMPANY.GETCONTACTLIST();

LIST<EMPLOYEE>EMPLIST = CONTACTLIST.GETEMPLOYEELIST();

FOR(EMPLOYEE EMP : EMPLIST){

SYSTEM.OUT.PRINTLN(EMP);

}

}

}
```

# ШАБЛОН ПРОКСИ (ВИРТУАЛНО(VIRTUAL))

COMPANY OBJECT CREATED...

COMPANY NAME: ABC COMPANY

COMPANY ADDRESS: BULGARIA

COMPANY CONTACT No.: +359-284589656

REQUESTING FOR CONTACT LIST

CREATING CONTACT LIST AND FETCHING LIST OF EMPLOYEES...

EMPLOYEE NAME: EMPLOYEE A, EMPLOYEEDESIGNATION: SE, EMPLOYEE SALARY: 2565.55

EMPLOYEE NAME: EMPLOYEE B, EMPLOYEEDESIGNATION: MANAGER, EMPLOYEE SALARY: 22574.0

EMPLOYEE NAME: EMPLOYEE C, EMPLOYEEDESIGNATION: SSE, EMPLOYEE SALARY: 3256.77

EMPLOYEE NAME: EMPLOYEE D, EMPLOYEEDESIGNATION: SSE, EMPLOYEE SALARY: 4875.54

EMPLOYEE NAME: EMPLOYEE E, EMPLOYEEDESIGNATION: SE, EMPLOYEE SALARY: 2847.01

# ШАБЛОН ПРОКСИ (PROTECTION PROXY)

```
PUBLIC INTERFACE STAFF {
PUBLIC BOOLEAN ISOWNER();
PUBLIC VOID SETREPORTGENERATOR(REPORTGENERATORPROXY REPORTGENERATOR);
}
```

# ШАБЛОН ПРОКСИ (PROTECTION PROXY)

```
PUBLIC CLASS EMPLOYEE IMPLEMENTS STAFF{
 PRIVATE REPORTGENERATORPROXY REPORTGENERATOR;

 @OVERRIDE
 PUBLIC VOID SETREPORTGENERATOR(REPORTGENERATORPROXY REPORTGENERATOR){
 THIS.REPORTGENERATOR = REPORTGENERATOR;
 }

 @OVERRIDE
 PUBLIC BOOLEAN ISOWNER() {

 RETURN FALSE;
 }

 PUBLIC STRING GENERATEDAILYREPORT(){

 TRY {
 RETURN REPORTGENERATOR.GENERATEDAILYREPORT();
 } CATCH (EXCEPTION E) {
 E.PRINTSTACKTRACE();
 }
 RETURN "";
 }
}
```

# ШАБЛОН ПРОКСИ (PROTECTION PROXY)

```
PUBLIC CLASS OWNER IMPLEMENTS STAFF {
 PRIVATE BOOLEAN ISOWNER=TRUE;
 PRIVATE REPORTGENERATORPROXY REPORTGENERATOR;
 @OVERRIDE
 PUBLIC VOID SETREPORTGENERATOR(REPORTGENERATORPROXY REPORTGENERATOR){
 THIS.REPORTGENERATOR = REPORTGENERATOR;
 }
 @OVERRIDE
 PUBLIC BOOLEAN ISOWNER(){
 RETURN ISOWNER;
 }
```

```
PUBLIC STRING GENERATEDAILYREPORT(){
 TRY {
 RETURN REPORTGENERATOR.GENERATEDAILYREPORT();
 } CATCH (EXCEPTION E) {
 E.PRINTSTACKTRACE();
 }
 RETURN "";
}
}
```

# ШАБЛОН ПРОКСИ (PROTECTION PROXY)

```
PUBLIC INTERFACE REPORTGENERATORPROXY {
PUBLIC STRING GENERATEDAILYREPORT();
}
```

# ШАБЛОН ПРОКСИ (PROTECTION PROXY)

```
import java.rmi.Naming;
import com.javacodegeeks.patterns.proxyPattern.RemoteProxy.ReportGenerator;

public class ReportGeneratorProtectionProxy implements ReportGeneratorProxy {
 ReportGenerator reportGenerator;

 Staff staff;

 public ReportGeneratorProtectionProxy(Staff staff) {
 this.staff = staff;
 }

 @Override
 public String generateDailyReport() {
 if (staff.isOwner()) {
 ReportGenerator reportGenerator = null;

 try {
 reportGenerator = (ReportGenerator) Naming.lookup("rmi://127.0.0.1/PizzaCoRemoteGenerator");

 return reportGenerator.generateDailyReport();
 } catch (Exception e) {
 e.printStackTrace();
 }

 return "";
 } else {
 return "NOT AUTHORIZED TO VIEW REPORT.";
 }
 }
}
```

# ШАБЛОН ПРОКСИ (PROTECTION PROXY)

```
PUBLIC CLASS TESTPROTECTIONPROXY {
 PUBLIC STATIC VOID MAIN(STRING[] ARGS) {
 OWNER OWNER = NEW OWNER();
 REPORTGENERATORPROXY REPORTGENERATOR = NEW REPORTGENERATORPROTECTIONPROXY(-
 OWNER);
 OWNER.SETREPORTGENERATOR(REPORTGENERATOR);
 EMPLOYEE EMPLOYEE = NEW EMPLOYEE();
 REPORTGENERATOR = NEW REPORTGENERATORPROTECTIONPROXY(EMPLOYEE);
 EMPLOYEE.SETREPORTGENERATOR(REPORTGENERATOR);
 SYSTEM.OUT.PRINTLN("FOR OWNER:");
 SYSTEM.OUT.PRINTLN(OWNER.GENERATEDAILYREPORT());
 SYSTEM.OUT.PRINTLN("FOR EMPLOYEE:");
 SYSTEM.OUT.PRINTLN(EMPLOYEE.GENERATEDAILYREPORT());
 }
}
```

# ШАБЛОН ПРОКСИ (PROTECTION PROXY)

FOR OWNER:

\*\*\*\*\*LOCATION X DAILY REPORT\*\*\*\*\*

LOCATION ID: 012

TODAY'S DATE: SUN SEP 14 13:28:12 IST 2014

TOTAL PIZZA SELL: 112

TOTAL SALE: \$2534

NET PROFIT: \$1985

\*\*\*\*\*

FOR EMPLOYEE:

NOT AUTHORIZED TO VIEW REPORT.

# ШАБЛОН ПРОКСИ (PROXY PATTERN)

- ОТДАЛЕЧЕНОТО ПРОКСИ - ПРЕДОСТАВЯ ЛОКАЛЕН ПРЕДСТАВИТЕЛ ЗА ОБЕКТ В РАЗЛИЧНО АДРЕСНО ПРОСТРАНСТВО.
- ВИРТУАЛНО ПРОКСИ - СЪЗДАВА "СКЪПИ" ОБЕКТИ ПРИ ПОИСКВАНЕ.
- ПРОКСИ ЗА ЗАЩИТА - КОНТРОЛИРА ДОСТЪПА ДО ОРИГИНАЛНИЯ ОБЕКТ. ПРОКСИ СЪРВЪРИТЕ ЗА ЗАЩИТА СА ПОЛЕЗНИ, КОГАТО ОБЕКТИТЕ ТРЯБВА ДА ИМАТ РАЗЛИЧНИ ПРАВА ЗА ДОСТЪП
- ДРУГИ ВИДОВЕ - CACHE PROXY/SERVER PROXY, FIREWALL PROXY, SYNCHRONIZATION PROXY, SMART REFERENCE PROXY, COUNTING PROXY